

KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 8 Laboratuvar Föyü

Veritabanı: SQLAlchemy ve Kalıcılık — Etkinlik/Bilet Sisteminin Belleğini Kalıcı Hâle Getirin

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Tahmini süre: 2 saat
Gereksinimler: Python 3.12 (asgari) ve Node.js 24 LTS kurulu bir bilgisayar, bir metin editörü (VS Code önerilir), Hafta 6–7'den tamamladığınız `sunucu` klasörü, Hafta 4–5, 7'den tamamladığınız `istemci-react` klasörü (bu hafta içinde hiçbir satırını değiştirmeyeceksiniz) — `sqlalchemy` paketini indirmek için internet bağlantısı gerekir

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim — Neden Önemli?

Hafta 7'nin sonunda React uygulamanız artık gerçek sunucudan veri çekiyordu, ama sunucunun kendi hafızası HÂLÂ geçiciydi: `uvicorn --reload` her yeniden başladığında girdiğiniz tüm etkinlik/koltuk/bilet verisi sıfırlanıyordu. Bugün iki saat sonra sunucunuzu `Ctrl+C` ile durdurup tekrar başlatacaksınız — ve `GET /etkinlikler` aynı verileri döndürmeye devam edecek, çünkü artık bir `bilet.db` dosyasında yazılı. Ayrıca Hafta 6'nın "aynı koltuğu iki kullanıcı aynı anda alamaz" kuralı (`409`), o zamana kadar yalnızca bir şans eseriydi — bugün bu kuralı gerçek bir veritabanı transaction'ıyla güvence altına alacaksınız: ikinci bir tarayıcı sekmesi bile gerekmeden, art arda gönderilen iki `curl` isteği bu korumayı ekranda gösterecek.

1 • Ön-öğretim — Ön Bilgi

Bu föy Hafta 8 ders notunun dört ünitesine dayanır: Ünite 1 (ORM Kavramı, Model-Tablo Eşlemesi ve Oturum), Ünite 2 (İlişkiler: Etkinlik–Koltuk–Bilet), Ünite 3 (Sorgular, Sayfalama ve Ham SQL Karşılığı) ve Ünite 4 (Transaction ve Eşzamanlı Rezervasyon). YG II'nin SQLite deneyimini (elle `CREATE TABLE/INSERT/ SELECT` yazıp dönen satırları kendi elinizle bir yapıya çevirme, ACID/transaction kavramı) hazır bilgi olarak kullanacaksınız; Hafta 6 Ünite 2'nin Pydantic `BaseModel/KamelModel` bilgisi bu haftanın ORM modelleriyle karşılaştırılacak; Hafta 6 Ünite 4'ün `Depends(get_depo)` deseni ve "Hafta 8'de bu yer tutucuyu değiştirecek" notu bugün gerçekleşecek; Hafta 6 Ünite 1'in sorgu parametresi kavramı (`?mekan=...`) bu haftanın `?sayfa=&boyut=` sayfalama parametrelerine doğrudan köprü kuracak.

Föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi kendi `sunucu` projenize **elle yazarak** tamamlayın, sonra terminaldeki çıktınızı *Beklenen sonuç* kutusuyla karşılaştırın. Adımlardaki kod kutuları `kod_ornekleri/hafta08/ sunucu/` klasöründeki referans dosyalardan gelir — bunlar hedef/"doğru cevap" niteliğindedir; siz bu kodu kopyala-yapıştır yapmak yerine kendi dosyalarınıza yazarsınız, sonucu terminalde karşılaştırırsınız. Birkaç adımda bir gelen *Kontrol Noktası*, o ana kadar yazdıklarınızı bir arada toparlamanıza yarar.

Bu hafta `istemci-react` klasörünüze tek bir satır bile dokunmayacaksınız: bugünün tüm değişikliği `sunucu` klasörünüzün içindedir. Hafta 4–5, 7'den tamamladığınız `istemci-react` klasörünüz bugün hiçbir satır değişmeden kalır — ön yüzünüz sunucunun içinde ne olduğunu (bellek

mi, dosya mı) hiç bilmez, aynı API sözleşmesiyle (rota adları, durum kodları, camelCase JSON alanları) konuşmaya devam eder. Föyün sonunda `istemci-react` klasöründe yalnızca `npm ci && npx tsc -b && npm run build` çalıştırıp kopyanın bozulmadığını doğrulayacaksınız — yeni bir kod satırı yazmayacaksınız.

1 ORM Kavramı, Model-Tablo Eşlemesi ve Oturum

(*bkz. Ders notu Hafta 8, Ünite 1 — ORM Kavramı, Model-Tablo Eşlemesi ve Oturum.*)

Adım 1: İki terminali açıp Hafta 6–7'nin hâlâ çalıştığını doğrulayın

Ne yapacağız? Terminal A'da `sunucu` klasörünüze girip sanal ortamı etkinleştirecek, mevcut test paketini çalıştıracak ve `requirements.txt` dosyanıza `sqlalchemy` bağımlılığını ekleyip kuracaksınız. Terminal B'de `istemci-react` klasöründe yalnızca `npm run build`'in HÂLÂ hatasız geçtiğini doğrulayacaksınız — bu hafta bu klasöre başka hiçbir şey yapmayacaksınız.

Neden? Bugünkü tüm adımlar Hafta 6–7'de kurduğunuz `sunucu` projesinin üzerine inşa edilecek — `app/modeller.py`, rota dosyaları ve testler aynen kalacak; değişen yalnızca `app/depo.py`'nin içinin bellekten (dict) veritabanına taşınmasıdır. Değişikliğe başlamadan önce mevcut testlerin kaç tane geçtiğini bilmek (11), gün sonunda kaç yeni test eklediğinizi (5 tane, toplam 16) izlemenizi sağlar.

Terminal

```
$ cd sunucu
$ source .venv/bin/activate
$ python -m pytest -q
```

Beklenen sonuç: Hafta 6–7'den kalan testlerin tamamı (11 test) hatasız geçer — bugünün sıfıra sıfırlanmış hâli budur.

Beklenen Çıktı

```
...........                                                  [100%]
11 passed, 2 warnings in 0.04s
```

```
fastapi~=0.141
uvicorn[standard]~=0.52
pydantic~=2.13
sqlalchemy~=2.0
```

Terminal

```
$ pip install -r requirements.txt -r requirements-dev.txt
```

Beklenen sonuç: Paket listesinde artık `sqlalchemy` de görünür; kurulum hatasız biter.

Beklenen Çıktı

```
Installing collected packages: ..., sqlalchemy, ..., fastapi
Successfully installed ... sqlalchemy-2.0.52 ...
```

Adım 1 (devamı)

```
>_ Terminal

$ cd ../istemci-react
$ npm run build
```

Beklenen sonuç: Derleme yine hatasız biter — `istemci-react` klasörünüze bugün hiçbir satır eklemeyeceksiniz, bu kontrol yalnızca Hafta 7'den kalan projenin sağlam olduğunu doğrular.

Adım 2: `app/veritabani.py` dosyasını yazın

Ne yapacağız? `sunucu` projenizde `app/veritabani.py` dosyasını oluşturup `engine` nesnesini, bildirimsel taban sınıfı (`Base`), oturum fabrikasını (`SessionLocal`) ve `get_db()` üreteç bağımlılığını yazacaksınız. Referans dosyanın en üstünde, `engine` tanımından hemen önce, iki `@event.listens_for` dekoratörü kullanan bir `sqlite_yazma_kilidi_uygula(engine)` yardımcı fonksiyonu da bulunur — bu fonksiyonu olduğu gibi kopyalayın; ne işe yaradığını Bölüm 4'te ayrıntılı öğreneceksiniz.

Neden? YG II'de SQLite'a elle bağlanıp SQL yazdınız; burada aynı SQLite dosyasına (`bilet.db`) SQLAlchemy'nin `engine` nesnesi üzerinden bağlanılır — `engine` uygulama başına genelde tek bir nesnedir, hangi veritabanı dosyasına/lehçeye bağlanılacağını bilir. `SessionLocal`, her istek için ayrı bir `Session` ("bir çalışma birimi") açıp kapatmanızı sağlar — bu, Hafta 6 Ünite 4'ün "Hafta 8'de `get_db_session()` bu yer tutucuyu değiştirecek" notunun gerçekleştiği yerdir: `get_db()` üreteç fonksiyonu, `yield`'den önceki bloğu isteği işleyen koddan önce, `finally` bloğunu (bir hata olsa bile) sonra çalıştırır.

```python
# echo=True açıldığında (Ünite 3) her çalıştırılan SQL cümlesi konsola
# basılır; günlük çalışmada gürültü yapmaması için varsayılan False'tur.
engine = create_engine("sqlite:///bilet.db", echo=False)
sqlite_yazma_kilidi_uygula(engine)
```

```python
class Base(DeclarativeBase):
    """Tüm ORM modellerinin (EtkinlikDB, KoltukDB, BiletDB) miras aldığı taban sınıf."
       ""
```

```python
SessionLocal = sessionmaker(bind=engine)
```

```python
def get_db() -> Generator[Session, None, None]:
    """Her istek için YENİ bir Session açar, istek bitince KAPATIR.

    `yield`'den ÖNCEKİ blok isteği İŞLEYEN koddan ÖNCE, `finally` bloğu
    isteği İŞLEYEN kod bittikten SONRA (hata OLSA bile) çalışır.
    """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Adım 2 (devamı)

Terminal

```
$ python3 -c "
from app.veritabani import Base, engine
Base.metadata.create_all(engine)
print('tamam')
"
$ ls bilet.db
```

Beklenen sonuç: Konsolda `tamam` yazısı görünür ve proje kökünde bir `bilet.db` dosyası oluşur — henüz hiçbir model `Base`'e kayıtlı olmadığından dosyanın içi boş bir şemadır, ama SQLite dosyasının kendisi artık diskte durur.

Beklenen Çıktı

```
tamam
bilet.db
```

İpucu

`bilet.db` dosyasını proje kökünüzde göreceksiniz ama bunu git'e commit'lemeyin — referans `.gitignore` dosyası bu satırı zaten içerir. Denemeleriniz sırasında dosya karıştıysa `rm bilet.db` ile silip bir sonraki `create_all` çağrısında sıfırdan oluşturabilirsiniz.

Adım 3: `app/orm_modeller.py`: `EtkinlikDB`'yi yazıp ilk kaydı ekleyip okuyun

Ne yapacağız? `app/orm_modeller.py` dosyasını oluşturup `EtkinlikDB` bildirimsel modelini yazacaksınız. Ardından bir Python oturumunda bir `EtkinlikDB` nesnesi oluşturup `db.add(...)` + `db.commit()` ile ekleyecek, sonra `db.get(EtkinlikDB, 1)` ile geri okuyacaksınız.

Neden? Her alanın `Mapped[tip]` tip ipucu + `mapped_column(...)` çağrısıyla tanımlanması, Hafta 6'nın Pydantic `BaseModel`'inin tip ipucundan doğrulama üretmesiyle kıyaslanabilir bir fikirdir: burada tip ipucundan bir sütun tanımı üretilir. `EtkinlikDB` bir veritabanı satırının Python temsilidir — YG II'de her tablo için tekrar tekrar yazdığınız "satırı nesneye çevirme" kodunun yerini bu sınıf tanımı alır.

```python
class EtkinlikDB(Base):
    """Bir `etkinlikler` satırının Python temsili."""

    __tablename__ = "etkinlikler"

    id: Mapped[int] = mapped_column(primary_key=True)
    baslik: Mapped[str]
    tarih: Mapped[str]
    mekan: Mapped[str]
    kapasite: Mapped[int]
    bilet_fiyati: Mapped[float]

    # Bir etkinlik SİLİNDİĞİNDE bağlı TÜM koltuklar (ve onların cascade'ine
    # bağlı biletleri) OTOMATİK silinir — Ünite 2'nin cascade silme kuralı.
    koltuklar: Mapped[list["KoltukDB"]] = relationship(
        back_populates="etkinlik", cascade="all, delete-orphan"
    )
```

Adım 3 (devamı)

```
>_ Terminal

$ python3
>>> from app.veritabani import Base, engine, SessionLocal
>>> from app.orm_modeller import EtkinlikDB
>>> Base.metadata.create_all(engine)   # yeni tablo: etkinlikler
>>> db = SessionLocal()
>>> etkinlik = EtkinlikDB(baslik="Bahar Konseri", tarih="2026-11-14",
...     mekan="KTÜ Kültür Merkezi", kapasite=100, bilet_fiyati=250)
>>> db.add(etkinlik)
>>> db.commit()
>>> db.refresh(etkinlik)
>>> etkinlik.id
1
>>> geri_okunan = db.get(EtkinlikDB, 1)
>>> geri_okunan.baslik, geri_okunan.mekan
('Bahar Konseri', 'KTÜ Kültür Merkezi')
```

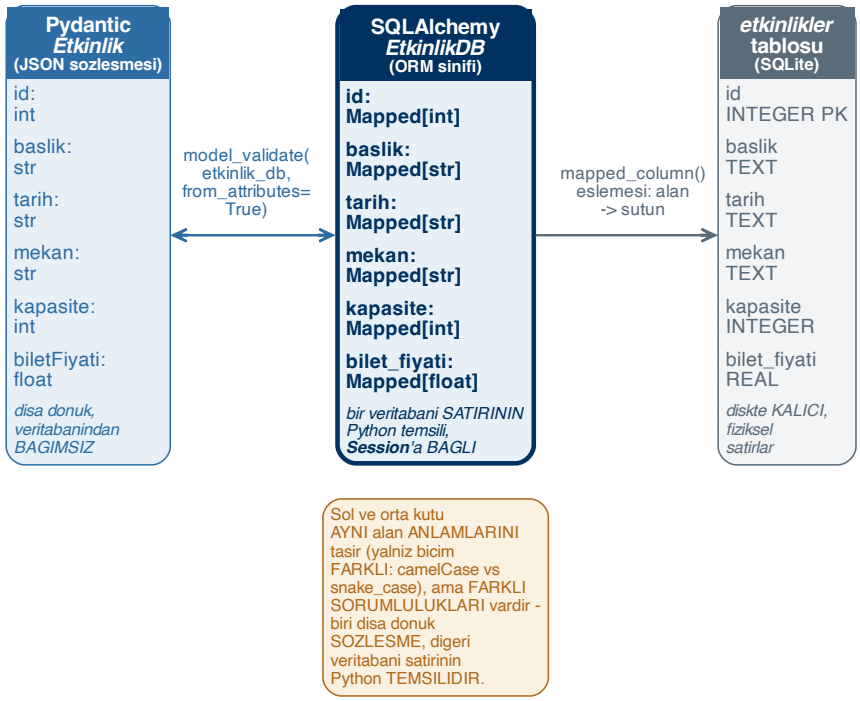
Beklenen sonuç: `etkinlik.id` otomatik olarak `1` atanır; `db.get` çağrısı eklediğiniz nesneyi geri döndürür — Python nesneniz artık diskte kalıcı bir satırdır, Python oturumunu kapatıp yeniden açsanız bile bu kayıt yerinde durur.

Adım 4: `model_validate(from_attributes=True)` köprüsünü deneyin

Ne yapacağız? Az önce eklediğiniz `EtkinlikDB` nesnesini, `app/modeller.py`'deki (Hafta 6'dan değişmeyen) `Etkinlik` Pydantic sınıfının `model_validate(from_attributes=True)` metoduyla bir Pydantic nesnesine çevireceksiniz.

Neden? `EtkinlikDB` bir `Session`'a bağlıdır, tembel yükleme yapabilir ve doğrudan JSON'a serileştirilmek için tasarlanmamıştır; `Etkinlik` ise dışa dönük, veritabanından tamamen bağımsız bir JSON sözleşmesidir. İki modeli tek bir sınıfta birleştirmek (Pydantic modelini doğrudan veritabanı satırı yapmak) katmanları karıştırır — Hafta 1 Ünite 3'ün "sunum/iş/veri katmanı" ayrımının bu haftaki somut karşılığı budur. Rota fonksiyonlarınız (Bölüm 2'de) bu köprüyü kullanarak veritabanından okunan ORM nesnesini dışa dönük Pydantic nesnesine çevirecek; FastAPI dönüş tipi hâlâ `Etkinlik`'tir — bu, rota imzalarınızın Hafta 6'dan değişmemesini sağlar.

Adım 4 (devamı)




```
Terminal
>>> from app.modeller import Etkinlik
>>> type(etkinlik).__name__
'EtkinlikDB'
>>> pydantic_etkinlik = Etkinlik.model_validate(etkinlik, from_attributes=True)
>>> type(pydantic_etkinlik).__name__
'Etkinlik'
>>> pydantic_etkinlik.model_dump_json(by_alias=True)
'{"baslik":"Bahar Konseri","tarih":"2026-11-14","mekan":"KTÜ Kültür
Merkezi","kapasite":100,"biletFiyati":250.0,"id":1}'
```

Beklenen sonuç: İlk nesnenin tipi `EtkinlikDB`, ikincisinin tipi `Etkinlik`'tir; `model_dump_json(by_alias=True)` çıktısında alan adları camelCase'dir (`biletFiyati`) — ORM katmanınızdaki `bilet_fiyati` (snake_case) ile dışa dönük JSON sözleşmeniz (Hafta 3'ten beri camelCase) burada buluşur.

Ortak Hata / Tuzak

`EtkinlikDB` nesnesini rota fonksiyonundan doğrudan (köprü olmadan) döndürmek — FastAPI'nin `response_model=Etkinlik` bildirimi bu durumda yine dönüştürme yapar, ama ORM nesnesinin tembel yüklenmiş (henüz erişilmemiş) alanlarına bu dönüşüm sırasında erişilmesi beklenmedik ek sorgulara ya da hatalara yol açabilir. `depo.py` metotlarınızın her zaman `model_validate(..., from_attributes=True)` ile açıkça dönüştürmesi bu riski ortadan kaldırır.

☑ Kontrol Noktası — devam etmeden doğrulayın

`app/veritabani.py`'yi yazıp proje kökünde bir `bilet.db` dosyası oluşturduysanız, `EtkinlikDB` ile bir kayıt ekleyip geri okuduysanız, `model_validate(from_attributes=True)` köprüsünün

ORM nesnesini Pydantic nesnesine çevirdiğini gördüyseniz, Ünite 1'in temelini kurmuşsunuz demektir — artık etkinlik/koltuk/bilet ilişkilerini kurabilirsiniz.

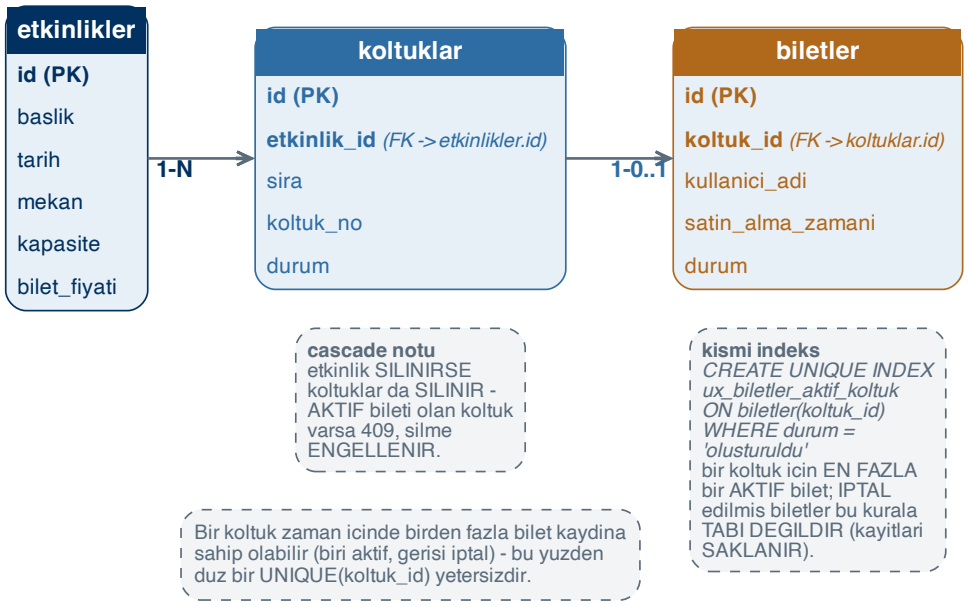
2 İlişkiler (Etkinlik–Koltuk–Bilet)

(*bkz. Ders notu Hafta 8, Ünite 2 — İlişkiler: Etkinlik–Koltuk–Bilet.*)

Adım 5: `KoltukDB`, `BiletDB` ve kısmi `UNIQUE` indeksi yazın

Ne yapacağız? `app/orm_modeller.py` dosyanıza `KoltukDB` ve `BiletDB` sınıflarını, `ForeignKey` ve `relationship/back_populates` tanımlarıyla ekleyeceksiniz. Dosyanın sonuna, bir koltuk için en fazla bir aktif bilet kaydına izin veren kısmi (koşullu) bir `UNIQUE` indeks de ekleyeceksiniz.

Neden? Bir etkinliğin birden fazla koltuğu olabilir, ama her koltuk yalnızca bir etkinliğe aittir — `etkinlik_id` yabancı anahtarı (YG II'nin `FOREIGN KEY` kavramının hatırlatılması) bu ilişkiyi kurar; `relationship/back_populates` iki modelin birbirini karşılıklı tanımasını sağlar — bir `etkinlik.koltuklar` yazdığınızda SQLAlchemy arka planda bir sorgu çalıştırır, elle bir `JOIN/WHERE etkinlik_id = ?` yazmanıza gerek kalmaz. İptal edilmiş biletler (Hafta 6'daki gibi) silinmez, `durum='iptal'` olarak saklanır — bu yüzden bir koltuğun zaman içinde birden fazla bilet kaydı olabilir (biri aktif, gerisi iptal) ve düz bir `UNIQUE(koltuk_id)` yetersizdir; kısmi indeks yalnızca `durum='olusturuldu'` olan satırlar arasında benzersizliği zorunlu kılar.




```python
class KoltukDB(Base):
    """Bir `koltuklar` satırının Python temsili — HER koltuk bir etkinliğe AİTTİR."""

    __tablename__ = "koltuklar"

    id: Mapped[int] = mapped_column(primary_key=True)
    etkinlik_id: Mapped[int] = mapped_column(ForeignKey("etkinlikler.id"))
    sira: Mapped[int]
    koltuk_no: Mapped[int]
    durum: Mapped[str] = mapped_column(default="bos")

    etkinlik: Mapped["EtkinlikDB"] = relationship(back_populates="koltuklar")
```

Adım 5 (devamı)

```
    biletler: Mapped[list["BiletDB"]] = relationship(
        back_populates="koltuk", cascade="all, delete-orphan"
    )
```

```
class BiletDB(Base):
    """Bir `biletler` satırının Python temsili.

    İptal edilmiş biletler SİLİNMEZ, `durum='iptal'` olarak SAKLANIR
    (denetim amaçlı) — bu yüzden bir koltuğun zaman içinde BİRDEN FAZLA
    bilet kaydı olabilir (biri aktif, gerisi iptal).
    """

    __tablename__ = "biletler"

    id: Mapped[int] = mapped_column(primary_key=True)
    koltuk_id: Mapped[int] = mapped_column(ForeignKey("koltuklar.id"))
    kullanici_adi: Mapped[str]
    satin_alma_zamani: Mapped[str]
    durum: Mapped[str] = mapped_column(default="olusturuldu")

    koltuk: Mapped["KoltukDB"] = relationship(back_populates="biletler")
```

```
# Bir koltuk için EN FAZLA bir AKTİF (durum='olusturuldu') bilet kaydına
# izin veren kısmi (koşullu) UNIQUE indeks — düz bir UNIQUE(koltuk_id)
# YETERSİZDİR, çünkü iptal edilmiş biletler silinmez ve bir koltuk zaman
# içinde birden fazla bilet kaydına sahip olabilir. Uygulama kontrolü
# (bkz. `app/depo.py` `bilet_ekle`) herhangi bir nedenle atlanırsa, bu
# indeks bir IntegrityError fırlatarak son bir güvence katmanı sağlar.
Index(
    "ux_biletler_aktif_koltuk",
    BiletDB.koltuk_id,
    unique=True,
    sqlite_where=(BiletDB.durum == "olusturuldu"),
)
```

Terminal

```
$ python3
>>> from app.veritabani import Base, engine, SessionLocal
>>> from app.orm_modeller import EtkinlikDB, KoltukDB
>>> Base.metadata.create_all(engine)   # yeni tablolar: koltuklar, biletler
>>> db = SessionLocal()
>>> etkinlik = db.get(EtkinlikDB, 1)
>>> for sira, no in [(1, 1), (1, 2), (1, 3)]:
...     koltuk = KoltukDB(
...         etkinlik_id=etkinlik.id, sira=sira, koltuk_no=no, durum="bos"
...     )
...     db.add(koltuk)
...
>>> db.commit()
>>> [(k.id, k.sira, k.koltuk_no) for k in etkinlik.koltuklar]
[(1, 1, 1), (2, 1, 2), (3, 1, 3)]
```

Beklenen sonuç: `etkinlik.koltuklar` bir Python listesi gibi davranır, ama arka planda SQLAlchemy sizin yerinize bir `SELECT` çalıştırır — üç koltuğu da (kimlikleriyle birlikte) geri döndürür.

Adım 6: `Depo` sınıfını ve etkinlik/koltuk metotlarını SQLAlchemy'e taşıyın

Ne yapacağız? `app/depo.py`'de `Depo.__init__`'in artık bir `Session` aldığını yazacak; `etkinlik_ekle`, `etkinlik_getir`, `etkinlik_listele`, `etkinlik_guncelle` ve `koltuk_ekle`, `koltuk_getir`, `koltuk_listele`, `koltuk_guncelle`, `koltuk_sil` metotlarını `select()`/`db.get()` sorgularıyla yeniden yazacaksınız.

Neden? Rota fonksiyonlarının gördüğü arayüz (metot adları/imzaları) değişmez — değişen yalnızca metotların içinin artık bir veritabanı `Session`'ı kullanmasıdır ("iyi bir API sözleşmesi deponun nasıl çalıştığını gizler"). `etkinlik_listele` metodunda `sayfa`/`boyut` parametrelerini zaten göreceksiniz — bunları şimdilik varsayılan değerleriyle (`sayfa=1`, `boyut=20`) çalışır kabul edin; nasıl çalıştıklarını Bölüm 3'te ayrıntılı inceleyeceksiniz.

```python
class Depo:
    """Etkinlik/Koltuk/Bilet kayıtlarını bir SQLAlchemy `Session`'ı üzerinden yönetir.
    """

    def __init__(self, db: Session) -> None:
        self.db = db
```

```python
def etkinlik_ekle(self, veri: EtkinlikOlustur) -> Etkinlik:
    etkinlik_db = EtkinlikDB(**veri.model_dump())
    self.db.add(etkinlik_db)
    self.db.commit()
    self.db.refresh(etkinlik_db)
    return Etkinlik.model_validate(etkinlik_db, from_attributes=True)

def etkinlik_getir(self, etkinlik_id: int) -> Etkinlik | None:
    etkinlik_db = self.db.get(EtkinlikDB, etkinlik_id)
    if etkinlik_db is None:
        return None
    return Etkinlik.model_validate(etkinlik_db, from_attributes=True)

def etkinlik_listele(
    self, sayfa: int = 1, boyut: int = 20, mekan: str | None = None
) -> list[Etkinlik]:
    stmt = select(EtkinlikDB).order_by(EtkinlikDB.id)
    if mekan is not None:
        stmt = stmt.where(EtkinlikDB.mekan == mekan)
    stmt = stmt.limit(boyut).offset((sayfa - 1) * boyut)
    etkinlikler_db = self.db.execute(stmt).scalars().all()
    return [
        Etkinlik.model_validate(e, from_attributes=True) for e in etkinlikler_db
    ]

def etkinlik_guncelle(
    self, etkinlik_id: int, veri: EtkinlikOlustur
) -> Etkinlik | None:
    etkinlik_db = self.db.get(EtkinlikDB, etkinlik_id)
    if etkinlik_db is None:
        return None
    for alan, deger in veri.model_dump().items():
        setattr(etkinlik_db, alan, deger)
    self.db.commit()
    self.db.refresh(etkinlik_db)
    return Etkinlik.model_validate(etkinlik_db, from_attributes=True)
```

```python
def koltuk_ekle(self, etkinlik_id: int, veri: KoltukOlustur) -> Koltuk:
    koltuk_db = KoltukDB(
```

Adım 6 (devamı)

```
        etkinlik_id=etkinlik_id, durum="bos", **veri.model_dump()
    )
    self.db.add(koltuk_db)
    self.db.commit()
    self.db.refresh(koltuk_db)
    return Koltuk.model_validate(koltuk_db, from_attributes=True)

def koltuk_getir(self, koltuk_id: int) -> Koltuk | None:
    koltuk_db = self.db.get(KoltukDB, koltuk_id)
    if koltuk_db is None:
        return None
    return Koltuk.model_validate(koltuk_db, from_attributes=True)

def koltuk_listele(
    self, etkinlik_id: int, durum: KoltukDurumu | None = None
) -> list[Koltuk]:
    stmt = select(KoltukDB).where(KoltukDB.etkinlik_id == etkinlik_id)
    if durum is not None:
        stmt = stmt.where(KoltukDB.durum == durum)
    koltuklar_db = self.db.execute(stmt).scalars().all()
    return [Koltuk.model_validate(k, from_attributes=True) for k in koltuklar_db]

def koltuk_guncelle(self, koltuk_id: int, durum: KoltukDurumu) -> Koltuk | None:
    koltuk_db = self.db.get(KoltukDB, koltuk_id)
    if koltuk_db is None:
        return None
    koltuk_db.durum = durum
    self.db.commit()
    self.db.refresh(koltuk_db)
    return Koltuk.model_validate(koltuk_db, from_attributes=True)

def koltuk_sil(self, koltuk_id: int) -> bool:
    koltuk_db = self.db.get(KoltukDB, koltuk_id)
    if koltuk_db is None:
        return False
    self.db.delete(koltuk_db)
    self.db.commit()
    return True
```

Beklenen sonuç: Dosya bu haliyle henüz çalıştırılamaz — `Depo`'nun bağımlılığı (`app/bagimliliklar.py`) hâlâ eski bir `dict` varsayıyor; bir sonraki adımda bu bağlantıyı kuracaksınız.

Ortak Hata / Tuzak

`etkinlik_listele` içinde `stmt.limit(boyut).offset(...)` satırını `where` koşulundan önce eklemek de çalışır (SQLAlchemy zincirleme çağrıları hangi sırada eklerseniz ekleyin doğru SQL'i üretir), ama okunabilirlik için filtre/sıralama/sayfalama sırasını referans dosyadaki gibi (önce `where`, sonra `limit`/`offset`) tutmanız önerilir.

Adım 7: `app/bagimliliklar.py` ve `app/main.py`'yi güncelleyip sunucuyu ilk kez başlatın

Ne yapacağız? `app/bagimliliklar.py`'deki `get_depo` fonksiyonunun artık `db: Session = Depends(get_db)` parametresi aldığını ve `Depo(db)` döndürdüğünü yazacaksınız. Ar-

Adım 7 (devamı)

dından `app/main.py`'ye, `uygulama_olustur` bloğundan sonra, `kok_rota` tanımından önce, `Base.metadata.create_all(engine)` çağrısını içeren bir blok ekleyeceksiniz. Son olarak sunucuyu başlatıp örnek veri yükleyeceksiniz.

Neden? Bu, zincirleme bir bağımlılıktır: `get_depo`'nun kendisi bir `Depends(get_db)` alır — rota fonksiyonlarınızdaki `depo: Depo = Depends(get_depo)` yazımı Hafta 6–7'den tek bir karakter değişmeden kalır; değişen yalnızca `get_depo`'nun içinde artık bir Python `dict` yerine bir veritabanı `Session` kullanılmasıdır ("aynı mekanizma, farklı bağımlılık"). `create_all` çağrısı sunucu başlarken, henüz yoksa, tüm modellerin (`EtkinlikDB`, `KoltukDB`, `BiletDB`) tablolarını oluşturur.

```python
def get_depo(db: Session = Depends(get_db)) -> Depo:
    """Rota fonksiyonlarına, bu isteğe ÖZEL bir veritabanı Session'ıyla
    kurulmuş bir Depo nesnesi enjekte eder.

    ZİNCİRLEME bağımlılık: get_depo'nun kendisi bir Depends(get_db) alır —
    rota fonksiyonlarındaki `depo: Depo = Depends(get_depo)` yazımı
    Hf.6-7'den TEK BİR karakter değişmeden kalır; değişen yalnızca
    get_depo'nun İÇİNDE artık bir Python dict YERİNE bir veritabanı
    Session kullanılmasıdır ("aynı mekanizma, farklı bağımlılık").
    """
    return Depo(db)
```

```python
# Uygulama BAŞLARKEN, henüz YOKSA, tüm ORM modellerinin (EtkinlikDB,
# KoltukDB, BiletDB) tablolarını bilet.db dosyasında oluşturur. Bu çağrı
# MEVCUT bir tabloyu ASLA GÜNCELLEMEZ — bir sütun eklerseniz create_all
# bunu MEVCUT tabloya yansıtmaz (dikkat: gerçek bir projede şema
# değişiklikleri Alembic gibi bir göç [migration] aracıyla sürüm sürüm
# kaydedilir; bu ders bu aracı KULLANMAZ, yalnızca ADINI anar).
Base.metadata.create_all(engine)
```

Terminal

```
$ rm -f bilet.db
$ uvicorn app.main:app --reload
```

Beklenen sonuç: Sunucu hatasız başlar; proje kökünde bir `bilet.db` dosyası yeniden oluşur (bu kez gerçek tablolarla).

Beklenen Çıktı

```
INFO:     Started server process [75799]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Terminal

```
$ python -m app.tohum
```

Beklenen sonuç: 2 örnek etkinlik ve 7 koltuk yüklenir — bu betik, sunucu kapalı olsa bile, ayrı bir terminalde çalıştırılabilir.

Adım 7 (devamı)

Beklenen Çıktı

```
2 etkinlik, 7 koltuk eklendi.
```

Dikkat

`create_all` mevcut bir tabloyu asla güncellemez — bir sütun eklerseniz, bu çağrı bunu mevcut tabloya yansıtmaz; şemayı sıfırdan kurmak için `bilet.db` dosyasını silip yeniden başlatmanız gerekir. Gerçek bir projede şema değişiklikleri (yeni sütun, tip değişikliği) Alembic gibi bir göç (*migration*) aracıyla sürüm sürüm kaydedilir; bu ders bu aracı kullanmaz, yalnızca adını anar.

Adım 8: Cascade silme kararını ve bilet rezervasyonunu ekleyip curl ile deneyin

Ne yapacağız? `app/depo.py`'ye `bilet_ekle` metodunu (koltuk kontrolü + doldur + bilet yaz tek transaction'da) ve `etkinlik_sil` metodunu (satılmış bilet varsa reddet, yoksa kademeli sil) yazacaksınız. `bilet_getir/bilet_listele/ bilet_iptal_et` metotlarını, `etkinlik/koltuk` metotlarında kullandığınız aynı `select()/db.get()` deseniyle tamamlayın — referans dosyada bulabilirsiniz. Ardından `app/rotalar/etkinlikler.py`'deki `etkinlik_sil` rotasını, sonucu `404/409/204`'e çevirecek şekilde güncelleyeceksiniz.

Neden? `DELETE /etkinlikler/{id}` çağrıldığında önce o etkinliğe bağlı koltuklardan herhangi birinin aktif (`durum= 'olusturuldu'`) bir bileti olup olmadığı kontrol edilir — varsa istek `409` ile reddedilir; yoksa etkinlik silinir ve `relationship`'teki `cascade="all, delete-orphan"` sayesinde bağlı tüm koltuklar otomatik olarak kademeli silinir — YG I/II'de bir ağaç/liste yapısını yıkarken alt düğümleri de tek tek serbest bırakma disiplininin ORM karşılığı budur. `bilet_ekle`'nin tek transaction'da (kontrol et + dolu yap + yaz) çalışması, iki adım arasında başka bir isteğin araya girebileceği bir boşluk bırakmaz — bu davranışın detayını Bölüm 4'te inceleyeceksiniz.

```python
def bilet_ekle(self, veri: BiletOlustur) -> Bilet:
    """Koltuk kontrolü + koltuğu doldurma + bilet ekleme TEK transaction'da.

    Bu Session'ın bağlı olduğu `engine`, her transaction'ı SQLite'ın
    `BEGIN IMMEDIATE`'iyle başlatır (`app/veritabani.py`) — aynı koltuğa
    eşzamanlı gelen ikinci istek, birincisi commit edene kadar burada
    BEKLER ve koltuğun ARTIK dolu olduğunu görüp `409` alır. Uygulama
    kontrolü (`durum == 'dolu'`) herhangi bir nedenle atlanırsa, kısmi
    UNIQUE indeks (`orm_aktif_koltuk_indeksi`) bir `IntegrityError`
    fırlatır — bu da `409`'a çevrilir; "tek kalem kuralı"nın veritabanı
    seviyesinde TEKRARLANMASI.
    """
    koltuk_db = self.db.get(KoltukDB, veri.koltuk_id)
    if koltuk_db is None:
        raise HTTPException(status_code=404, detail="Koltuk bulunamadı")
    if koltuk_db.durum == "dolu":
        raise HTTPException(status_code=409, detail="Bu koltuk zaten satılmış")

    koltuk_db.durum = "dolu"
    bilet_db = BiletDB(
        koltuk_id=veri.koltuk_id,
        kullanici_adi=veri.kullanici_adi,
        satin_alma_zamani=datetime.now().isoformat(),
        durum="olusturuldu",
```

Adım 8 (devamı)

```
    )
    self.db.add(bilet_db)

    try:
        self.db.commit()
    except IntegrityError:
        self.db.rollback()
        raise HTTPException(
            status_code=409, detail="Bu koltuk zaten satılmış"
        ) from None
    self.db.refresh(bilet_db)
    return Bilet.model_validate(bilet_db, from_attributes=True)
```

```
def etkinlik_sil(self, etkinlik_id: int) -> str:
    """Etkinliği SİLER (cascade) ya da neden silinemediğini bildirir.

    Dönüş: `"silindi"` | `"bulunamadi"` | `"aktif_bilet_var"`. Bağlı
    koltuklardan HERHANGİ BİRİNİN aktif (`durum='olusturuldu'`) bir
    bileti varsa silme REDDEDİLİR; yoksa `relationship`'teki
    `cascade="all, delete-orphan"` sayesinde bağlı TÜM koltuklar (ve
    varsa iptal edilmiş biletleri) OTOMATİK olarak kademeli silinir.
    """
    etkinlik_db = self.db.get(EtkinlikDB, etkinlik_id)
    if etkinlik_db is None:
        return "bulunamadi"

    aktif_bilet_var = any(
        bilet.durum == "olusturuldu"
        for koltuk in etkinlik_db.koltuklar
        for bilet in koltuk.biletler
    )
    if aktif_bilet_var:
        return "aktif_bilet_var"

    self.db.delete(etkinlik_db)
    self.db.commit()
    return "silindi"
```

```
@router.delete("/{etkinlik_id}", status_code=204)
def etkinlik_sil(etkinlik_id: int, depo: Depo = Depends(get_depo)) -> None:
    sonuc = depo.etkinlik_sil(etkinlik_id)
    if sonuc == "bulunamadi":
        raise HTTPException(status_code=404, detail="Etkinlik bulunamadı")
    if sonuc == "aktif_bilet_var":
        raise HTTPException(
            status_code=409,
            detail="Bu etkinliğin satılmış biletleri var, önce iptal edilmelidir",
        )
```

Terminal

```
$ curl -s -i -X POST http://127.0.0.1:8000/biletler \
  -H "Content-Type: application/json" \
  -d '{"koltukId": 1, "kullaniciAdi": "Ayse Yilmaz"}'
```

Beklenen sonuç: `201 Created` döner — 1. koltuğa ait bir bilet oluşturulur, koltuk `'dolu'`

Adım 8 (devamı)

durumuna geçer.

Beklenen Çıktı

```
HTTP/1.1 201 Created
content-type: application/json

{"koltukId":1,"kullaniciAdi":"Ayse Yilmaz","id":1,
"satinAlmaZamani":"2026-09-07T16:18:57.303398",
"durum":"olusturuldu"}
```

Terminal

```
$ curl -s -i -X DELETE http://127.0.0.1:8000/etkinlikler/1
$ curl -s -i -X DELETE http://127.0.0.1:8000/etkinlikler/2
$ curl -s -i http://127.0.0.1:8000/etkinlikler/2
```

Beklenen sonuç: 1. etkinlik (satılmış bileti var) `409` ile reddedilir; 2. etkinlik (satılmamış) `204` ile silinir; sonra 2. etkinliği GET etmek `404` döner.

Beklenen Çıktı

```
HTTP/1.1 409 Conflict
{"detail":"Bu etkinliğin satılmış biletleri var, önce iptal edilmelidir"}

HTTP/1.1 204 No Content

HTTP/1.1 404 Not Found
{"detail":"Etkinlik bulunamadı"}
```

Kontrol Noktası — devam etmeden doğrulayın

`KoltukDB`/`BiletDB` ilişkilerini kurduysanız, `etkinlik.koltuklar` ile ilişkili kayıtları okuduysanız, sunucunuzu başlatıp `bilet.db` + tohum verisini oluşturduysanız, satılmış bileti olan bir etkinliği silmenin `409`, olmayanı silmenin `204` döndüğünü gözlemlediyseniz, Ünite 2'nin temelini kurmuşsunuz demektir — artık sorgularınızı sayfalayıp arkada üretilen SQL'i inceleyebilirsiniz.

3 Sorgular, Sayfalama ve Ham SQL Karşılığı

(*bkz. Ders notu Hafta 8, Ünite 3 — Sorgular, Sayfalama ve Ham SQL Karşılığı.*)

Adım 9: Liste uç noktalarına `sayfa`/`boyut` sorgu parametrelerini ekleyin

Ne yapacağız? `app/rotalar/etkinlikler.py`'deki `etkinlik_liste` rotasını, `sayfa: int = 1` ve `boyut: int = 20` sorgu parametrelerini alıp Bölüm 2'de yazdığınız `depo.etkinlik_listele`'ye iletecek şekilde güncelleyeceksiniz.

Neden? Bu, Hafta 6 Ünite 1'in sorgu parametresi kavramının (`?mekan=...`) doğrudan devamıdır: mevcut liste uç noktasına (dizi dönen `GET` rotası) yeni, isteğe bağlı sorgu parametreleri eklenir — dönüş tipi hâlâ bir dizidir, bu yüzden OpenAPI şeması geriye uyumlu kalır. 500 etkinlik olsa tek seferde hepsini döndürmek hem sunucuyu hem tarayıcıyı yorar;

Adım 9 (devamı)

`limit/offset` bu yükü küçük parçalara böler.

```python
@router.get("", response_model=list[Etkinlik])
def etkinlik_liste(
    mekan: str | None = None,
    sayfa: int = 1,
    boyut: int = 20,
    depo: Depo = Depends(get_depo),
) -> list[Etkinlik]:
    return depo.etkinlik_listele(sayfa=sayfa, boyut=boyut, mekan=mekan)
```

```bash
# Sayfalama: en fazla 2 kayıt döner (sayfa=1, boyut=2).
curl -i "$BASE_URL/etkinlikler?sayfa=1&boyut=2"
```

Terminal

```
$ curl -s "http://127.0.0.1:8000/etkinlikler?sayfa=1&boyut=1"
```

Beklenen sonuç: En fazla bir kayıt döner — 7 koltuğu bulunan "Bahar Konseri" etkinliği (kimliği 1).

Beklenen Çıktı

```
[{"baslik":"Bahar Konseri","tarih":"2026-11-14",
"mekan":"KTÜ Kültür Merkezi","kapasite":100,
"biletFiyati":250.0,"id":1}]
```

İpucu

Toplam kayıt sayısını (sayfalama dışı, tüm eşleşen kayıt sayısı) istemciye bildirmek isterseniz, `func.count` ile hesaplanan bu sayıyı bir `X-Toplam-Kayit` yanıt başlığı (*header*) olarak dönebilirsiniz — bu haftanın "Kendiniz ekleyin" görevinde bunu kendiniz uygulayacaksınız.

Adım 10: `echo=True` ile ORM'in ürettiği SQL'i gözlemleyin

Ne yapacağız? Ayrı bir demo betiğinde, `create_engine(..., echo=True)` ile açtığınız bir `engine` üzerinden `select(EtkinlikDB).where(EtkinlikDB.mekan == 'KTÜ Kültür Merkezi')` sorgusunu çalıştıracak, konsolda basılan gerçek SQL cümlesini `sorgular.sql` dosyasındaki elle yazılmış karşılığıyla karşılaştıracaksınız.

Neden? `echo=True` Açıkken her `db.execute(...)` çağrısının arkasında çalıştırdığı gerçek SQL cümlesi (parametreleriyle birlikte) konsola basılır. Bu, YG II'de öğrendiğiniz "ne olduğunu bil" disiplininin ORM katmanındaki karşılığıdır — ORM'in ürettiği SQL, YG II'de elle yazdığınız SQL'den biçim olarak farklı görünebilir (parametre yer tutucuları, sütun listesi sırası), ama fonksiyonel olarak aynı sonucu üretir; bu bir sorun değildir.

```sql
-- GET /etkinlikler?mekan=Salon+A karşılığı
SELECT etkinlikler.id, etkinlikler.baslik, etkinlikler.tarih,
       etkinlikler.mekan, etkinlikler.kapasite, etkinlikler.bilet_fiyati
FROM etkinlikler
WHERE etkinlikler.mekan = 'Salon A'
ORDER BY etkinlikler.id;
```

Adım 10 (devamı)

Terminal

```
$ python3
>>> from sqlalchemy import create_engine, select
>>> from sqlalchemy.orm import sessionmaker
>>> from app.orm_modeller import EtkinlikDB
>>> engine = create_engine("sqlite:///bilet.db", echo=True)
>>> Session = sessionmaker(bind=engine)
>>> db = Session()
>>> stmt = select(EtkinlikDB).where(EtkinlikDB.mekan == "KTÜ Kültür Merkezi")
>>> sonuc = db.execute(stmt).scalars().all()
```

Beklenen sonuç: Konsolda `BEGIN`, ardından `WHERE etkinlikler.mekan = ?` içeren gerçek bir `SELECT` cümlesi ve parametre değeri görünür — bu, `sorgular.sql`'deki elle yazdığınız `sql_etkinlik_listele` kesitiyle aynı mantığı taşır, yalnızca ORM bunu sizin yerinize, Python nesnelerinden üretmiştir.

Beklenen Çıktı

```
INFO sqlalchemy.engine.Engine BEGIN (implicit)
INFO sqlalchemy.engine.Engine SELECT etkinlikler.id,
etkinlikler.baslik, etkinlikler.tarih, etkinlikler.mekan,
etkinlikler.kapasite, etkinlikler.bilet_fiyati
FROM etkinlikler
WHERE etkinlikler.mekan = ?
INFO sqlalchemy.engine.Engine [generated in 0.00008s]
('KTÜ Kültür Merkezi',)
```

Bir etkinliği koltuklarıyla birlikte tek sorguda çekmenin (Ünite 2'nin `selectinload` ile istekli yüklemesinin elle yazılmış `JOIN` karşılığı) SQL'i şudur:

```sql
-- Bir etkinliği koltuklarıyla BİRLİKTE tek sorguda çekme (selectinload'ın
-- elle yazılmış JOIN karşılığı — Ünite 2'nin istekli yükleme örneği)
SELECT etkinlikler.id, etkinlikler.baslik, koltuklar.id AS koltuk_id,
     koltuklar.sira, koltuklar.koltuk_no, koltuklar.durum
FROM etkinlikler
JOIN koltuklar ON koltuklar.etkinlik_id = etkinlikler.id
WHERE etkinlikler.id = 1;
```

İpucu

ORM'in `select()` API'si rahat ifade edemediği karmaşık bir rapor/performans-kritik bir sorgu gerektiğinde, `db.execute( text("SELECT ...") )` bir kaçış kapısı olarak kullanılabilir — genel kural: "ORM'in `select()` API'si yeterliyse onu kullanın, yetmiyorsa `text()`'e düşün."

Kontrol Noktası — devam etmeden doğrulayın

Liste uç noktanız `?sayfa=&boyut=` parametrelerini kabul ediyorsa, `echo=True` çıktısında gerçek bir SQL cümlesi gördüyseniz ve bunu `sorgular.sql`'deki elle yazılmış karşılığıyla karşılaştırdıysanız, Ünite 3'ün temelini kurmuşsunuz demektir — artık eşzamanlı rezervasyon korumasını ekleyebilirsiniz.

4 Transaction ve Eşzamanlı Rezervasyon

(*bkz. Ders notu Hafta 8, Ünite 4 — Transaction ve Eşzamanlı Rezervasyon.*)

Adım 11: `bilet_ekle`'nin tek-transaction korumasını ve `BEGIN IMMEDIATE`'i inceleyin

Ne yapacağız? Bölüm 2'de yazdığınız `bilet_ekle` metoduna ve Bölüm 1'de `app/veritabani.py`'nin başına kopyaladığınız `sqlite_yazma_kilidi_uygula(engine)` çağrısına yeniden bakacak, bu ikisinin birlikte nasıl bir koruma sağladığını `sorgular.sql`'deki elle yazılmış transaction karşılığıyla karşılaştıracaksınız.

Neden? Hafta 6'nın bellekteki `dict`'inde "aynı koltuğu iki kullanıcı aynı anda alırsa" sorunu hiç gözlemlenemedi — tek bir Python süreci, gelen istekleri pratikte sırayla işliyordu. Bu hafta veri bir dosyada yaşadığından bu yarış durumu (*race condition*) artık gerçek bir tehdittir. Çözüm, "kontrol et + dolu yap + yaz" üç adımının tek bir `Session` ömrü, tek bir `db.commit()` sınırında yapılmasıdır — adımlar ayrı ayrı `commit` edilseydi, iki commit arasındaki boşlukta başka bir isteğin araya girip aynı koltuğu alması mümkün olurdu. SQLite'ın varsayılan transaction'ı (`BEGIN DEFERRED`) bir yazma işlemine kadar kilit almaz; `sqlite_yazma_kilidi_uygula`'nın gönderdiği `BEGIN IMMEDIATE` ile transaction başlar başlamaz bir yazma kilidi alınır — aynı koltuğa eşzamanlı gelen ikinci istek, birincisi commit edene kadar tam burada bekler. Uygulama kontrolü (`if koltuk_db.durum == "dolu"`) herhangi bir nedenle atlanırsa, Bölüm 2'de eklediğiniz kısmi `UNIQUE` indeks bir `IntegrityError` fırlatır — bu da `409`'a çevrilir: "tek kalem kuralı"nın veritabanı seviyesinde tekrarlanması.

```sql
-- POST /biletler işleyicisinin TEK transaction'da yaptığı üç adımın
-- (kontrol et + dolu yap + yaz) elle SQL karşılığı
BEGIN IMMEDIATE;

SELECT id, durum FROM koltuklar WHERE id = 1;
-- Uygulama kodu burada durum = 'bos' mu diye kontrol eder.

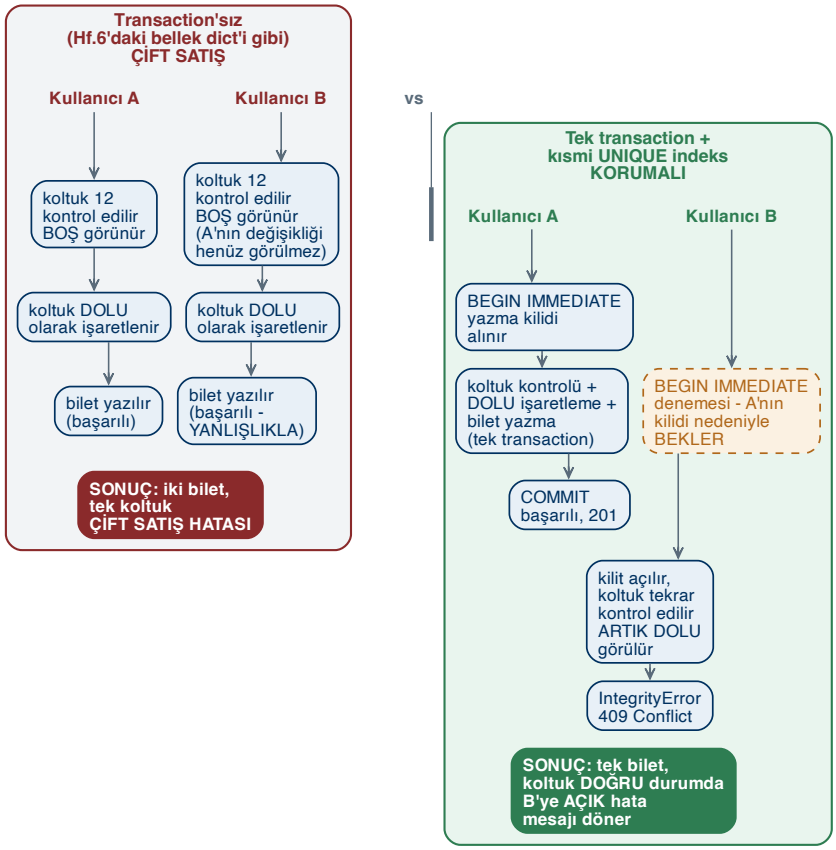
UPDATE koltuklar SET durum = 'dolu' WHERE id = 1;

INSERT INTO biletler (koltuk_id, kullanici_adi, satin_alma_zamani, durum)
VALUES (1, 'Ayşe Yılmaz', '2026-11-14T10:00:00', 'olusturuldu');

COMMIT;
```

```sql
-- Bir koltuk için EN FAZLA bir AKTİF bilet kaydına izin veren kısmi
-- (koşullu) UNIQUE indeks — app/orm_modeller.py'deki Index(...) tanımının
-- ürettiği DDL
CREATE UNIQUE INDEX ux_biletler_aktif_koltuk
ON biletler (koltuk_id)
WHERE durum = 'olusturuldu';
```

Adım 11 (devamı)



Beklenen sonuç: `BEGIN IMMEDIATE` ile `COMMIT` arasında, üç adımın (kontrol et, güncelle, yaz) tek bir yazma kilidi altında yürüdüğünü fark edersiniz — Şekil'deki iki senaryo (transaction'sız çift satış hatası vs. tek transaction/`UNIQUE` indeksli ikincinin `409` alması) arasındaki farkı görürsünüz.

İpucu

Gerçek bir üretim sisteminde "`database is locked`" hatası alan bir isteğin, küçük bir gecikmeyle yeniden denenmesi (*retry*) geçici kilit çakışmalarını tolere etmenin yaygın bir yoludur — bu haftanın kodu bunu uygulamaz, yalnızca fikri anıyoruz.

Adım 12: `tests/test_transaction.py` dosyasını yazıp tüm test paketini çalıştırın

Ne yapacağız? `sunucu` projenizde `tests/test_transaction.py` dosyasını oluşturup `test_ardisik_rezervasyon_409` testini yazacak, ardından tüm test paketini çalıştıracaksınız.

Neden? Bu test aynı koltuk için ardışık iki `POST /biletler` isteği gönderip ikinci isteğin `409` döndüğünü ve sunucuda tek bir bilet kaydı olduğunu doğrular — ikinci isteğin yazdığı hiçbir kayıt yoktur, veritabanı tutarlı kalır. `tests/conftest.py`'deki fixture'lar (Hafta 6–7'den güncellendi) her testi ayrı, belleğe kurulu (`:memory:`) bir SQLite veritabanıyla başlatır — testleriniz proje kökünüzdeki `bilet.db` dosyasına hiç dokunmaz.

```
def test_ardisik_rezervasyon_409(client):
    """Aynı koltuk için ARDIŞIK iki POST /biletler isteği — ikincisi 409 döner."""
    _, koltuk = _etkinlik_ve_koltuk_olustur(client)
```

Adım 12 (devamı)

```
    ilk = client.post(
        "/biletler", json={"koltukId": koltuk["id"], "kullaniciAdi": "Ayşe Yılmaz"}
    )
    assert ilk.status_code == 201

    ikinci = client.post(
        "/biletler", json={"koltukId": koltuk["id"], "kullaniciAdi": "Mehmet Demir"}
    )
    assert ikinci.status_code == 409
    assert "satılmış" in ikinci.json()["detail"]

    # Sunucuda TEK bir bilet vardır — ikinci isteğin YAZDIĞI hiçbir kayıt YOK.
    biletler = client.get("/biletler").json()
    assert len(biletler) == 1
```

Terminal

```
$ python -m pytest -q
```

Beklenen sonuç: Hafta 6–7'nin 11 testi + bu haftanın 5 yeni testi (sayfalama, cascade/409 silme x2, ardışık rezervasyon 409, opsiyonel eşzamanlı threading testi) = 16 test hatasız geçer.

Beklenen Çıktı

```
................                                         [100%]
16 passed, 2 warnings in 0.21s
```

Adım 12 (devamı)

Dikkat

opsiyonel bir ileri deneme olarak, `test_esz_zamanli_ rezervasyon_threading` testi `threading` modülüyle gerçekten eşzamanlı iki isteği tetikleyip yalnızca birinin `201`, diğerinin `409` aldığını doğrular — bu test gerçek yarış durumunu simüle eder, öncekinden daha kırılgandır (zamanlama duyarlıdır), bu yüzden zorunlu değildir:

```
def test_esz_zamanli_rezervasyon_threading(tmp_path):
    """OPSİYONEL: GERÇEKTEN eşzamanlı iki istek — yalnızca biri 201, diğeri 409
        alır.

    Diğer testlerin paylaştığı tek-bağlantılı `:memory:` veritabanının
    AKSİNE, bu test kendi dosya tabanlı SQLite veritabanını (ayrı gerçek
    bağlantılarla) kurar — GERÇEK bir SQLite dosya kilidinin iki ayrı
    bağlantı arasında NASIL çalıştığını sınar; bu yüzden zamanlamaya
    DUYARLIDIR ve diğer testlerden AYRI, kendi fixture'ını kurar.
    """
    db_yolu = tmp_path / "esz_zamanli_test.db"
    # `timeout=30`: ikinci bağlantının BEGIN IMMEDIATE'i, ilki commit edene
    # kadar en çok 30 s bekler (SQLite busy timeout; varsayılan 5 s, yüklü bir
    # makinede iki iş parçacığı yarışırken yetmeyebilir ve test kararsızlaşır).
    engine = create_engine(
        f"sqlite:///{db_yolu}",
        connect_args={"check_same_thread": False, "timeout": 30},
    )
    sqlite_yazma_kilidi_uygula(engine)
    Base.metadata.create_all(engine)
    session_fabrikasi = sessionmaker(bind=engine)

    def get_db_gecersiz_kil():
        db = session_fabrikasi()
        try:
            yield db
        finally:
            db.close()

    app.dependency_overrides[get_db] = get_db_gecersiz_kil
    try:
        with TestClient(app) as istemci:
            _, koltuk = _etkinlik_ve_koltuk_olustur(istemci)

            def rezervasyon_yap(kullanici_adi: str):
                return istemci.post(
                    "/biletler",
                    json={"koltukId": koltuk["id"], "kullaniciAdi": kullanici_adi},
                )

            with ThreadPoolExecutor(max_workers=2) as havuz:
                gorev_1 = havuz.submit(rezervasyon_yap, "Ayşe Yılmaz")
                gorev_2 = havuz.submit(rezervasyon_yap, "Mehmet Demir")
                cevap_1 = gorev_1.result()
                cevap_2 = gorev_2.result()
    finally:
        app.dependency_overrides.pop(get_db, None)

    durum_kodlari = sorted([cevap_1.status_code, cevap_2.status_code])
    assert durum_kodlari == [201, 409]
```

Adım 13: Uçtan uca deneyin: sunucuyu yeniden başlatıp kalıcılığı ve tam akışı doğrulayın

Ne yapacağız? Sunucunuzu `Ctrl+C` ile durdurup tekrar başlatacak, `GET /etkinlikler`'in aynı verileri döndürdüğünü doğrulayacak, sayfalamayı tekrar deneyecek, bir koltuğa ardışık iki bilet isteği gönderip `409`'u gözlemleyecek, son olarak satılmış bileti olan ve olmayan birer etkinliği silerek cascade kuralını uçtan uca doğrulayacaksınız.

Neden? Bu adım, bugün yazdığınız dört ünitenin (ORM/oturum, ilişkiler/ cascade, sayfalama/SQL, transaction) hepsini tek bir akışta birleştirir — Etkinlik/Bilet Sisteminin belleği nihayet kalıcı olmuştur, sunucuyu kapatıp açmanız artık hiçbir veriyi kaybettirmez.

Terminal

```
$ Ctrl+C
$ rm -f bilet.db
$ uvicorn app.main:app --port 8000 &
$ python -m app.tohum
$ curl -s "http://127.0.0.1:8000/etkinlikler?sayfa=1&boyut=1"
$ curl -s -i -X POST http://127.0.0.1:8000/biletler \
  -H "Content-Type: application/json" \
  -d '{"koltukId": 1, "kullaniciAdi": "Ayse Yilmaz"}'
$ curl -s -i -X POST http://127.0.0.1:8000/biletler \
  -H "Content-Type: application/json" \
  -d '{"koltukId": 1, "kullaniciAdi": "Mehmet Demir"}'
$ curl -s -i -X DELETE http://127.0.0.1:8000/etkinlikler/1
$ curl -s -i -X DELETE http://127.0.0.1:8000/etkinlikler/2
$ curl -s -i http://127.0.0.1:8000/etkinlikler/2
```

Beklenen sonuç: Sırasıyla: sayfalanmış liste tek kayıt döner; ilk bilet isteği `201`; ikinci (aynı koltuk) `409`; 1. etkinliği silme (satılmış bileti var) `409`; 2. etkinliği silme (satılmamış) `204`; 2. etkinliği tekrar GET etme `404`.

Beklenen Çıktı

```
[{"baslik":"Bahar Konseri","tarih":"2026-11-14",
"mekan":"KTÜ Kültür Merkezi","kapasite":100,
"biletFiyati":250.0,"id":1}]

HTTP/1.1 201 Created
{"koltukId":1,"kullaniciAdi":"Ayse Yilmaz","id":1,
"satinAlmaZamani":"2026-09-07T16:18:57.303398",
"durum":"olusturuldu"}

HTTP/1.1 409 Conflict
{"detail":"Bu koltuk zaten satılmış"}

HTTP/1.1 409 Conflict
{"detail":"Bu etkinliğin satılmış biletleri var, önce iptal edilmelidir"}

HTTP/1.1 204 No Content

HTTP/1.1 404 Not Found
{"detail":"Etkinlik bulunamadı"}
```

Kontrol Noktası — devam etmeden doğrulayın

Sunucunuzu yeniden başlatıp verinin yerinde durduğunu gördüyseniz, sayfalanmış bir liste aldıysanız, aynı koltuğa ardışık iki istekte ikincisinin `409` aldığını, satılmış bileti olan bir etkinliğin `409`, olmayanın `204` ile silindiğini gözlemlediyseniz, `python -m pytest -q` çıktısında `16 passed` gördüyseniz, bu haftanın dört ünitesini de uygulamışsınız demektir — Etkinlik/Bilet Sisteminin belleği artık kalıcıdır ve rezervasyon kuralı gerçek bir transaction'a dayanır.

Adım 14: Kendiniz ekleyin: toplam kayıt sayısını `X-Toplam-Kayit` başlığında dönün

Ne yapacağız? `app/rotalar/etkinlikler.py`'deki `etkinlik_liste` rota fonksiyonuna bir `Response` parametresi ekleyecek; `depo`'nuzda (ya da rota içinde) `func.count` ile tüm eşleşen kayıtların (sayfalama dışı) toplam sayısını hesaplayacak ve bu sayıyı `response.headers["X-Toplam-Kayit"]` üzerinden metin olarak döndüreceksiniz.

Neden? `sayfa`/`boyut` parametreleri istemciye yalnızca o sayfadaki kayıtları verir — istemcinin "toplam kaç sayfa var" diye hesaplayabilmesi için toplam kayıt sayısını ayrıca bilmesi gerekir. Bu bilgiyi gövdeye (JSON dizisine) eklemek dönüş tipini (hâlâ bir dizi) bozar; bir yanıt başlığında taşımak OpenAPI şemasını değiştirmeden aynı bilgiyi taşır.

Beklenen sonuç: `curl -i "http://127.0.0.1:8000/etkinlikler?sayfa=1&boyut=1"` çalıştırdığınızda yanıt başlıkları arasında `x-toplam-kayit: 2` (ya da o andaki gerçek toplam) satırını görürsünüz — gövdedeki dizi hâlâ en fazla `boyut` kadar eleman içerir.

> **İpucu**
>
> `select(func.count()).select_from(EtkinlikDB)` (gerekirse aynı `where` koşuluyla) size toplam satır sayısını verir — sayfalama için kullandığınız `stmt`'in `limit`/`offset` eklenmeden önceki hâlini bu sayım sorgusunda yeniden kullanabilirsiniz.

Sorun Giderme

Belirti	Olası neden	Çözüm
`ImportError: cannot import name 'DeclarativeBase'`	SQLAlchemy 1.x sürümü kurulu (`requirements.txt` güncellenmeden eski bir `.venv` kullanılıyor)	`pip show sqlalchemy` ile sürümü kontrol edip `2.0` ile başladığından emin olun; gerekirse `.venv`'i silip yeniden oluşturun
Sunucu başlarken `no such table: etkinlikler` hatası	`app/main.py`'ye `create_all` çağrısı eklenmemiş, ya da `orm_modeller.py` dosyası hiç içe aktarılmamış (bu durumda modeller `Base`'e hiç kaydolmaz)	`app/main.py`'nin başında `from app import orm_modeller` satırının ve `tablo_olustur` bloğunun var olduğunu doğrulayın
`DELETE /etkinlikler/{id}` her zaman `204` dönüyor, satılmış bileti olan bir etkinlik bile silinebiliyor	`etkinlik_sil` metodundaki `durum == "olusturuldu"` kontrolü `koltuk.biletler` ilişkisine hiç erişmiyor (ör. Yanlış alan adı karşılaştırılıyor)	`app/depo.py`'deki `depo_etkinlik_sil_cascade` kesitini referans dosyayla satır satır karşılaştırıp `aktif_bilet_var` ifadesinin doğru koşulu kontrol ettiğini doğrulayın
Aynı koltuğa ardışık iki `POST /biletler` isteği gönderdiğinizde ikisi de `201` dönüyor (ikinci istek `409` almıyor)	`bilet_ekle` içindeki `koltuk_db.durum == "dolu"` kontrolü ilk isteğin `db.commit()`'inden önce çalışmış olabilir, ya da Bölüm 2'de kısmi `UNIQUE` indeks hiç eklenmemiş	`app/orm_modeller.py`'deki `orm_aktif_koltuk_indeksi` kesitinin dosyanın sonunda, modül düzeyinde (bir sınıfın içinde değil) tanımlı olduğunu doğrulayın; `bilet.db` dosyasını silip sunucuyu yeniden başlatarak indeksin taze oluşturulmasını sağlayın
`python -m pytest -q` çalıştırırken proje kökünde bir `bilet.db` dosyası oluşuyor ve testler bu dosyayı paylaşıyor gibi davranıyor	`tests/conftest.py`'deki `client` fixture'ı `get_db` bağımlılığı için bir geçersiz kılma (*override*) atanmamış, testler HÂLÂ üretim `get_db`'sini kullanıyor	`tests/conftest.py`'deki `test_istemci_fixture` kesitinde `get_db` için bir geçersiz kılma atandığını doğrulayın

5 · İzleme — Aktarım — Başka Nerede?

Bugün Etkinlik/Bilet Sisteminin belleğini bellekten (dict) gerçek bir veritabanına (SQLite) taşıdınız: `DeclarativeBase`/`Mapped`/`mapped_column` ile ORM modellerinizi yazdınız, `ForeignKey`/`relationship` ile Etkinlik–Koltuk–Bilet zincirini kurdunuz, cascade silme kuralını kesinleştirdiniz, sayfalanmış sorgular yazıp `echo=True` ile arkada üretilen SQL'i okudunuz ve tek-transaction rezervasyon korumasını gerçek bir yarış durumuna karşı doğruladınız. Bugün kurduğunuz `engine`/`Session`/`get_db` deseni Hafta 10'un `Kullanici` tablosunda, Hafta 13'ün arka plan görevlerinde aynı şekilde kullanılacak; `relationship`/`selectinload` deseniniz Hafta 15'in performans ünitesinde N+1 sorununu çözerken derinleşecek; bugünkü tek-transaction disiplininiz Hafta 13'ün WebSocket bildirimlerinde ("koltuk durumu değiştiğinde herkese haber ver") ve Hafta 15'in katmanlı mimarisinde aynı ilkeyle sürecek. Gelecek hafta (Hafta 9) Ara Değerlendirme'dir — yeni konu yoktur, Hafta 1–8'i sentezleyen bir arabölüm ve entegre bir lab sizi bekliyor. Hafta 10'da ise `kullaniciAdi` alanınız HÂLÂ herkesin istediğini yazabildiği düz bir metin olmaktan çıkıp gerçek bir `Kullanici` tablosuna, şifre özetine ve bir oturuma (JWT) bağlanacak — bugün kurduğunuz `Base`/`engine`/`Session` deseni o tabloyu eklerken aynen yeniden kullanılacaktır.